

UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

RELAZIONE CONCLUSIVA DI INGEGNERIA
DEL SOFTWARE

Sistema di gestione issue in progetti software

Authors:
Ubaldo Musto N86005304
Giandomenico Iameo
N86002856

Supervisor:
Prof. Porfirio TRAMONTANA
Co-Supervisor:
Prof. Bernardo BREVE

Versione 1.0.0
Prima versione del prodotto software, contenente le funzionalità di base per la gestione collaborativa di issue in progetti software

Dipartimento di Ingegneria Elettrica e delle Tecnologie dell'Informazione
Corso di Laurea Triennale in Informatica

14 novembre 2026

COPYRIGHT ©2025, BY UBALDO AND GIANDOMENICO
ALL RIGHTS RESERVED.

Indice

1 Specifica dei requisiti **1**
1.1 Scopo del documento dei requisiti 1
1.2 Scopo del prodotto 1
1.3 Riferimenti 2
1.4 Descrizione del resto del documento 2

2 Descrizione generale **4**
2.1 Prospettiva del prodotto 4
2.2 Funzioni del prodotto 4
2.2.1 Requisiti funzionali utente 4
2.2.2 Diagrammi di caso d'uso 5
2.3 Caratteristiche utente 9
2.4 Presupposti e dipendenze 9
2.4.1 Dipendenze tecnologiche 9
2.4.2 Dipendenze sull'ambiente dell'utente 10

3 Requisiti specifici **11**
3.1 Requisiti funzionali 11
3.1.1 Template di Cockburn 12
3.1.2 Prototipazione visuale via Mock-up 13
3.2 Requisiti non funzionali 14
3.2.1 Requisiti del prodotto 14
3.2.2 Requisiti organizzativi 15

4 Design del sistema **17**
4.1 Modello architetturale client-server 17
4.1.1 Livello di presentazione (*client-tier*) 17
4.1.2 Livello della logica business (*application-tier*) 18
4.1.3 Livello di gestione dei dati (*database-tier*) 18
4.1.4 Perché conviene separare i livelli 18
4.2 Decomposizione del sistema 19
4.2.1 Front-end 19
4.2.1.1 Pattern MVP-*Passive* 20
4.2.1.2 Stratificazione del sottosistema 22
4.2.2 Back-end: pattern Controller-Service-Repository 22

4.2.3 Back-end: Repository pattern 23
4.3 Comunicazione tra front-end e back-end 23
4.3.1 Autenticazione 24
4.3.2 Interfaccia delle API 24
4.4 Deployment 25
4.5 Criteri di design 26
4.6 Scelte tecnologiche 27
4.6.1 Spring Boot 27
4.6.2 MySQL e Flyway 27
4.6.3 Docker 28
4.6.4 JavaFX 28
4.6.5 Linguaggi e strumenti 28

5 Verifica e convalida **30**
5.1 Progettazione dei casi di test 30
5.2 Descrizione dei casi di testi in JUnit 30
5.2.1 Metodo `selectToggleFilter()` 31
5.2.2 Metodo `setDate()` 34
5.3 Testing lato back-end 37
5.3.1 Test plan: creazione di una issue 37
5.3.2 Metodo `creaIssue()` 37
5.3.2.1 Criterio di copertura delle classi di equivalenza 42
5.4 Test plan: creazione di una issue 42
5.4.1 Obiettivo 42
5.4.2 Oggetti del test 42
5.4.3 Ambiente di testing 42
5.4.4 Criteri di superamento 42

Capitolo 1

Specifica dei requisiti

1.1 Scopo del documento dei requisiti

Il documento *SRS*, riportato nei primi tre capitoli, si propone di accogliere in forma organica una descrizione di tutte le specifiche tecniche e funzionali che caratterizzano il sistema per la gestione di issue in progetti software.
Esso è stato redatto seguendo le indicazioni dello standard IEEE/ANSI 830-1998 (IEEE 1998) per i documenti dei requisiti.

1.2 Scopo del prodotto

Il prodotto software che si vuole realizzare prende il nome di BugBoard26, una piattaforma che consentirà a un team di sviluppo di segnalare problemi relativi a un progetto, monitorare lo stato, assegnarli a membri del team e tenere traccia delle attività di risoluzione. Il sistema deve fornire le seguenti funzionalità:

1. Il sistema deve fornire un meccanismo di autenticazione semplice e sicuro, basato su email e password. Le informazioni gestite dall'applicazione sono critiche per l'azienda, ed è fondamentale preservarne l'integrità e la segretezza. Il sistema viene fornito con un account da amministratore già attivo, con credenziali di default. Un amministratore può creare ulteriori utenze, specificando una email, una password, e indicando se quell'utenza sarà "normale" oppure "di amministrazione".

2. Tutti gli utenti autenticati possono segnalare una issue indicando almeno un titolo e una descrizione. Alcuni utenti potrebbero voler specificare anche una priorità e sarebbe gradita la possibilità di allegare un'immagine. Le issue possono essere di diverso tipo: question (per richieste di chiarimenti), bug (per segnalare malfunzionamenti), documentation (per segnalare problemi relativi alla documentazione), e feature (per indicare la richiesta o il suggerimento di nuove funzionalità). Le issue create sono inizialmente nello stato "todo".

3. Il sistema deve offrire una vista riepilogativa delle issue, con la possibilità di filtrare o ordinare i risultati in base a criteri come tipologia, stato, priorità o altri parametri rilevanti.
4. Ogni bug dovrebbe poter essere assegnato (da un amministratore) a un membro del team. Quando viene assegnato un bug a un utente, quest'ultimo riceve una notifica. Gli utenti possono visualizzare i bug loro assegnati.
5. È necessaria una funzione di ricerca che consenta di trovare bug in base a parole chiave.
6. Il sistema dovrebbe suggerire automaticamente a chi assegnare un bug, basandosi sul carico di lavoro corrente degli utenti.

1.3 Riferimenti

Qui di seguito sono elencati i documenti e gli indirizzi web a cui fa riferimento la presente specifica dei requisiti, nonché i link al codice sorgente e al repository Docker

- Slide del corso di Ingegneria del software 2025/2026 fornite dai professori P.Tramontana e B.Breve.
- Ian, Sommerville. *Ingegneria del software*. 8ª edizione, Pearson Paravia Bruno Mondadori S.p.A., 2007.
- Modello IEEE/ANSI 830-1998 per strutturare il documento dei requisiti.
- Enciclopedia online Wikipedia.
- Repository del back-end: la guida all'installazione è nel file `README.md`.
- Repository del front-end: la guida all'installazione è nel file `README.md` e i pacchetti eseguibili sono nella sezione Release.

1.4 Descrizione del resto del documento

La restante parte del presente documento è suddivisa in due ulteriori capitoli che forniscono una descrizione sempre più dettagliata delle funzionalità del prodotto software in questione. Il capitolo immediatamente successivo, *Descrizione generale*, è composto dai seguenti paragrafi:

1. *Prospettiva del prodotto:* descrive il contesto o ambiente operativo in cui sarà utilizzato il prodotto software in questione. Verrà specificata l'architettura generale, se il prodotto è un membro di una famiglia di prodotti, un sostituto di altri sistemi oppure un prodotto completamente autonomo.
2. *Funzioni del prodotto:* illustra i principali servizi (o funzionalità) del prodotto corredati da diagrammi. Il livello di dettaglio usato in questo paragrafo è a un alto livello; i dettagli delle funzionalità saranno forniti del capitolo 3.

3. *Caratteristiche utente:* esplicità le conoscenze richieste per poter utilizzare il prodotto in questione e fornisce una descrizione generale degli utilizzatori previsti dal sistema.

4. *Presupposti e dipendenze:* illustra le condizioni che il sistema presume siano verificate al momento del suo avvio, durante la sua esecuziione e dopo la terminazione. Verranno specificati anche *elementi esterni* (altri sistemi, persone o componenti) da cui il prodotto dipende e che sono necessari per il suo corretto funzionamento.

Nel terzo capitolo, *Requisiti specifici*, verranno illustrate le funzionalità che il prodotto software realizza. Il capitolo è composto dai seguenti paragrafi:

1. *Requisiti funzionali:* fornisce una descrizione dettagliata dei casi d'uso che verranno presentati nel capitolo 2. Tali requisiti indicano come il sistema dovrebbe reagire a particolari input o come dovrebbe comportarsi in particolari situazioni.

2. *Requisiti non funzionali:* esplicita i vincoli sui servizi o sulle funzioni offerti dal sistema. In questa sezione, i requisiti imposti sono stati divisi secondo la *classificazione di Sommerville*.

Capitolo 2

Descrizione generale

2.1 Prospettiva del prodotto

Il prodotto software sarà un'applicazione basata su un'architettura a tre livelli, composta da due macro-componenti indipendenti: una parte di *back-end* e una di *front-end*.

Back-end
Avrà la responsabilità della gestione centralizzata dello stato del sistema, garantendo che tutte le informazioni e i dati persistenti siano conservati e aggiornati unicamente su di esso.

Front-end
Costituirà l'interfaccia utente che si appoggierà ai servizi REST esposti dal back-end.

2.2 Funzioni del prodotto

In questo paragrafo sono stati elencati i requisiti funzionali utente, dedotti dalle funzionalità descritte nel primo capitolo, e descritti mediante i diagrammi di caso d'uso. Il paragrafo è suddiviso in due parti:

> Nella *prima parte* verrà fornito un elenco dei requisiti funzionali dedotti dalle specifiche del committente. Tali requisiti sono descritti con un linguaggio chiaro, comprensibile anche a parsone senza un'approfonddita conoscenza tecnica.
>
> Nella *seconda parte* verranno presentati i diagrammi di caso d'uso, i quali saranno associati a uno o più requisiti.

2.2.1 Requisiti funzionali utente

RF1 **Il sistema deve offrire un meccanismo di autenticazione.**

RF2 **Il sistema deve poter permettere a ogni account *Amministratore* di creare ulteriori utenti, sia normali che di tipo amministratore, specificando le loro credenziali di accesso.**

RF3 **Gli utenti possono segnalare issue, con la possibilità di specificare un titolo, una descrizione, una priorità, una immagine e il tipo.**

RF4 **Il sistema deve permettere la visualizzazione di tutte le issue segnalate, con la possibilità di filtrare e ordinare i risultati.**

RF5 **Il sistema deve permettere, solo a un *Amministratore*, di assegnare bug e notificare un membro del team.**

RF6 **Il sistema deve consentire a un *utente normale* di ricevere notifiche riguardanti i bug.**

RF7 **Ogni *utente normale* può visualizzare i propri bug assegnati.**

RF8 **Il sistema deve fornire una funzione di ricerca per trovare issue.**

RF9 **Il sistema dovrebbe suggerire automaticamente a chi assegnare un bug.**

2.2.2 Diagrammi di caso d'uso

Qui di seguito vengono illustrati i diagrammi di caso d'uso che modellano i requisiti sopra citati.

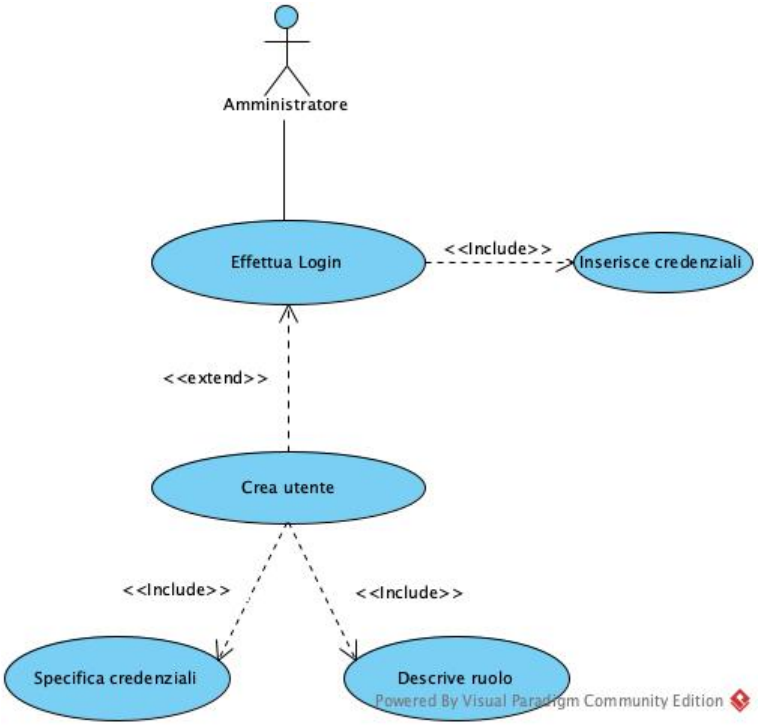


Figura 2.1: Modellazione requisiti *RF1, RF2*

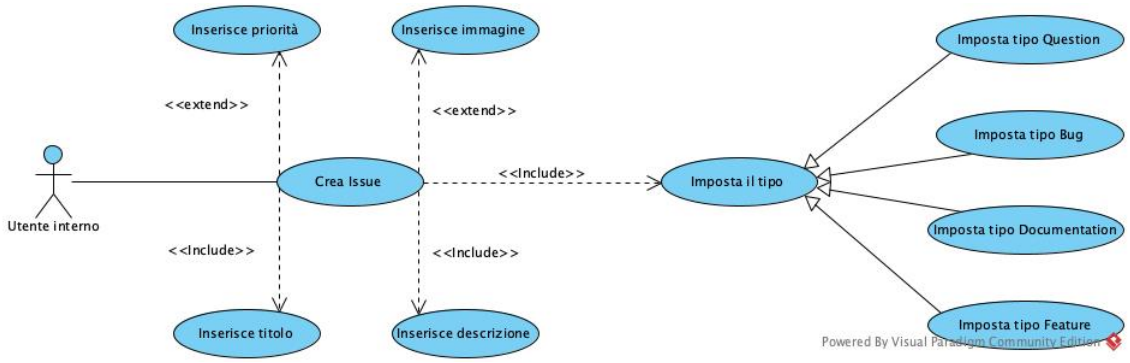


Figura 2.2: Modellazione requisito *RF3*

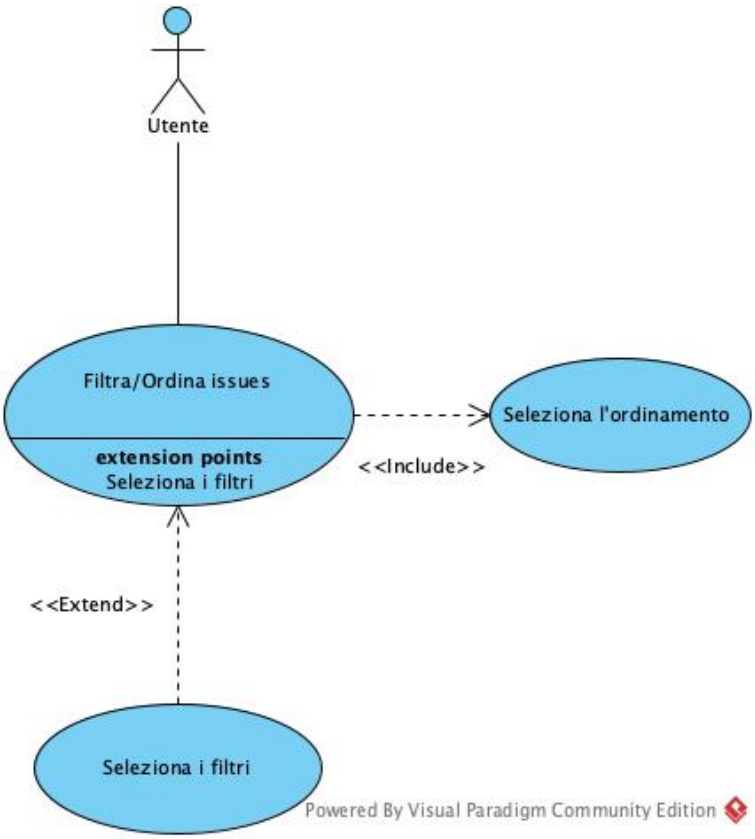


Figura 2.3: Modellazione requisito *RF4*

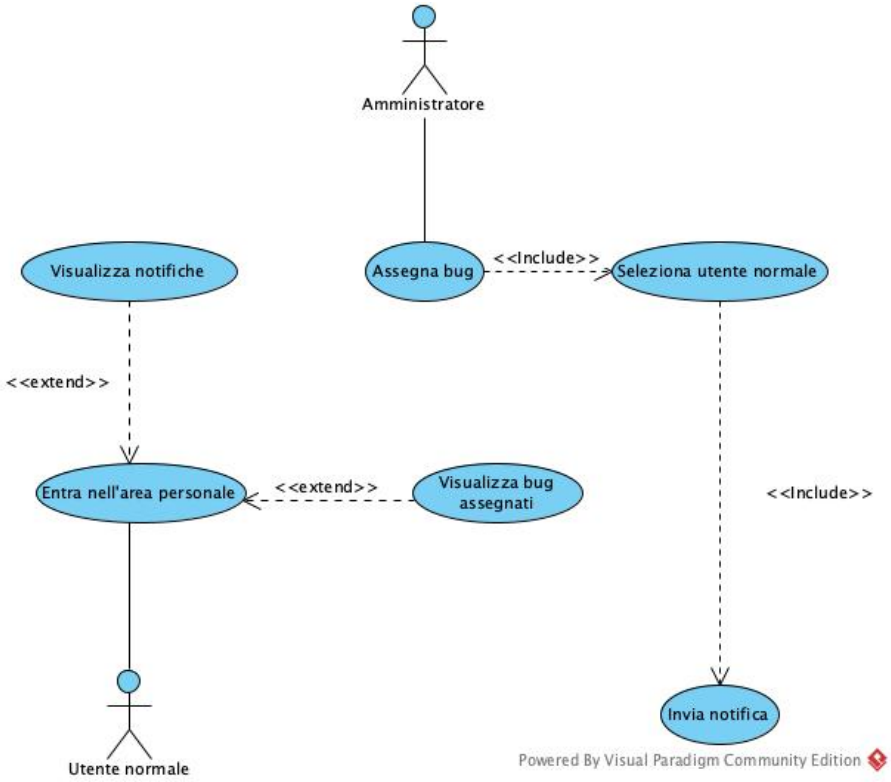


Figura 2.4: Modellazione requisiti *RF5, RF6, RF7*

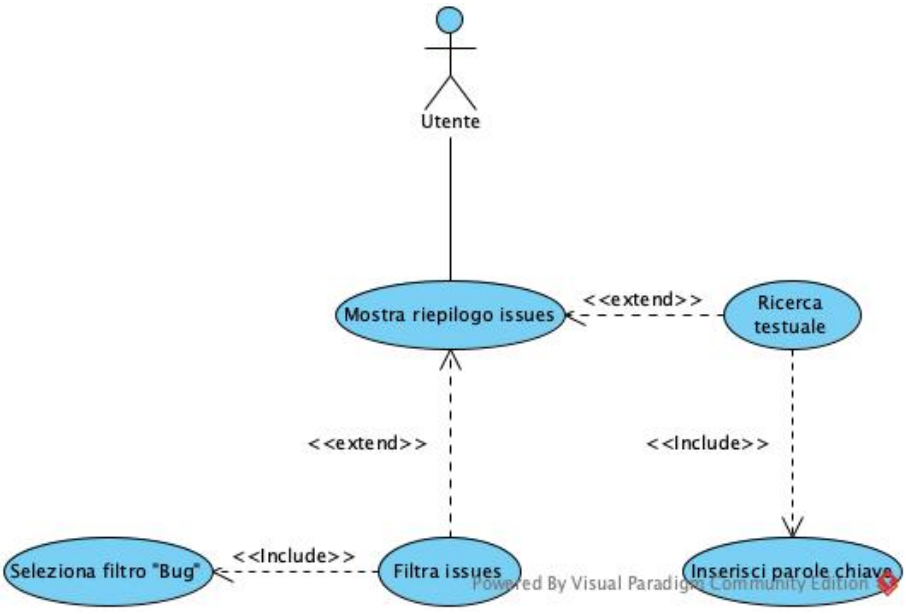


Figura 2.5: Modellazione requisiti *RF4, RF8*

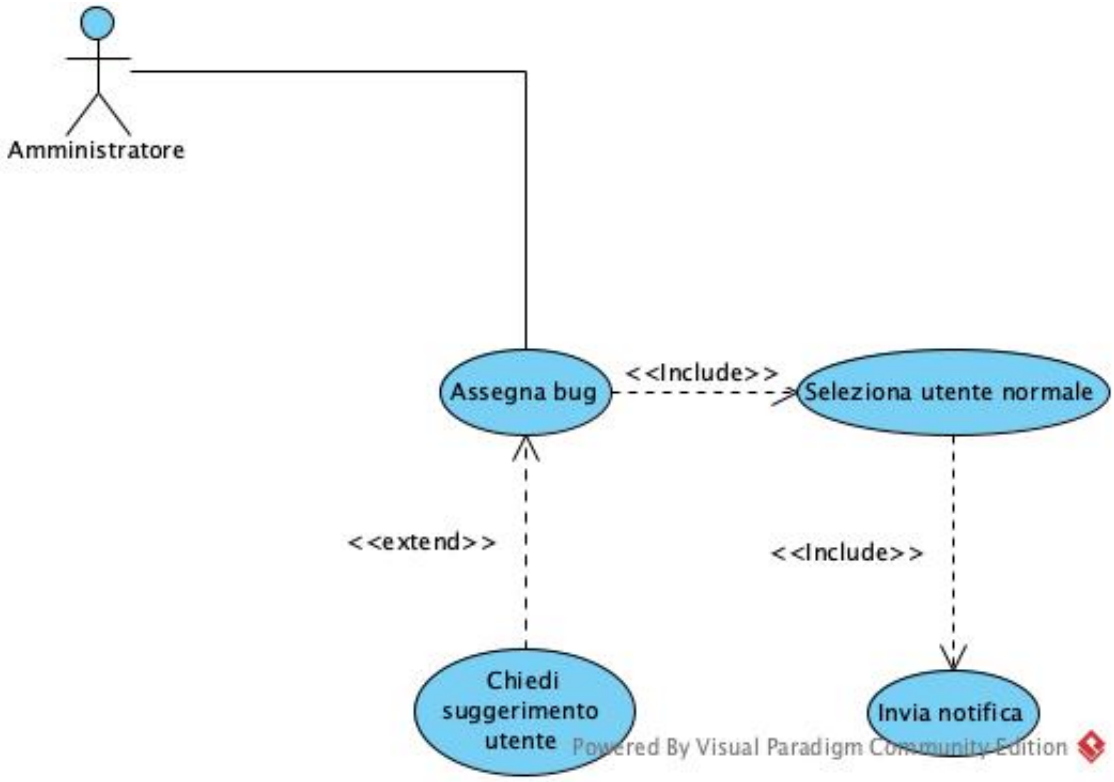


Figura 2.6: Modellazione requisiti *RF5,RF9*

2.3 Caratteristiche utente

L'utilizzo di questo prodotto software da parte degli utenti richiede che questi ultimi abbiano una buona conoscenza del dominio applicativo in cui il software opera. Per tale motivo, gli utenti del sistema sono stati divisi in due categorie in funzione del ruolo ricoperto.

1. *utente normale*
2. *utente mministratore*

Gli *utenti amministratori* sono coloro che utilizzano tutte le funzionalità messe a disposizione dal sistema.

Gli *utenti normali*, invece, utilizzano un sottoinsieme limitato delle funzionalità, pur possedendo una conoscenza approfondita del dominio applicativo. È il caso, in particolare, dei membri di un team di sviluppo. Questi devono essere in grado di riconoscere e saper descrivere chiaramente i loro problemi (bug, suggerimento di nuove funzionalità, ecc.).

Per qualsiasi tipologia di utente, una scarsa conoscenza del dominio applicativo può portare all'incapacità di riconoscere e descrivere correttamente tali problemi, con conseguente perdita di tempo e potenziali fraintendimenti. Nello specifico, le definizioni e i ruoli dei singoli attori del sistema sono:

Amministratore
Si riferisce a un utente che ha accesso completo al sistema, senza limitazione alcuna. Tra le operazioni che può effettuare figurano la gestione degli utenti e la gestione dei bug (come ad esempio, l'assegnazione di un bug a un membro del team).

Utente normale
Chiamato anche utente *regolare*, si riferisce a un utente senza privilegi speciali. Il suo ruolo è quello di segnalare problemi, proporre miglioramenti o fare richieste funzionali.

Tali account vengono assegnati normalmente ai membri di un team di sviluppo, i quali possono anche visualizzare i bug assegnati in un'apposita sezione.

2.4 Presupposti e dipendenze

2.4.1 Dipendenze tecnologiche

- *Containerizzazione del back-end*: Il committente desidera che una componente del software, ovvero la parte di *back-end*, sia distribuita utilizzando tecnologie di containerizzazione come *Docker*.
  - *Dipendenze*: Si assume che le macchine server supportino l'esecuzione di *container Docker*, includendo un demone *Docker* installato correttamente e configurato.

2.4.2 Dipendenze sull'ambiente dell'utente

- *Connettività di rete*: L'architettura di sistema prevede che il *front-end* comunichi con il *back-end* anche quando questo risiede in una rete esterna.
  - *Dipendenze*: Qualora le parti si trovino in due reti separate, si assume che gli utenti finali dispongano di una connessione Internet stabile e affidabile per poter utilizzare l'applicazione senza interruzioni. La qualità dell'esperienza utente dipenderà direttamente dalle prestazioni della rete.

Capitolo 3

Requisiti specifici

Il presente capitolo cerca di fornire una descrizione più dettagliata dei requisiti funzionali utente presentati nel capitolo 2, nonché dei requisiti non funzionali imposti.

Anche se, in teoria, i requisiti di sistema dovrebbero semplicemente descrivere i vincoli operativi e non come il sistema dovrebbe essere progettato, il livello di dettaglio adottato ha reso impossibile escludere tutte le informazioni sulla progettazione.

3.1 Requisiti funzionali

Il presente paragrafo si presta a descrivere in maniera più dettagliata i casi d'uso presentati nel precedente capitolo. Per la descrizione testuale di tali casi d'uso si è deciso di adottare un formato proposto da Alistar Cockburn e chiamato *template di Cockburn*. Il motivo di tale scelta è legato alla sua struttura tabellare, che facilita la comprensione delle interazioni tra attori e sistema, offrendo una rappresentazione esaustiva di tutti gli eventi.

Si fa presente al lettore che le tabelle seguenti contengono riferimenti a *mock-up*. Per una maggiore comprensione si raccomanda di consultare questi artefatti quando vengono citati. Questo paragrafo è suddiviso nei seguenti sottoparagrafi:

1. *Template di Cockburn*: Presentazione del *template di Cockburn* applicato a un *caso d'uso* significativo: *Filtra issues*.
2. *Prototipazione visuale via Mock-up*: I prototipi selezionati rispecchiano fedelmente l'interfaccia utente realizzata. Ciò ha permesso di individuare agevolmente sia le incongruenze nel design grafico, sia quelle relative alla logica operativa del sistema.

3.1.1 Template di Cockburn

Use case #1	*Filtra/Ordina issues*		
Goal in Context	*Consente all'utente di ordinare e filtrare le issue in base a diversi criteri*		
Preconditions	*L'utente deve possedere un proprio account*		
Success End Condition	*Il sistema restituisce i risultati della ricerca avanzata*		
Failed End Condition	*Il sistema non produce nessun risultato di ricerca*		
Primary Actor	Utente		
Trigger	*L'utente clicca sul pulsante* `Filtra`		
Main Scenario	**Step n.**	**Utente**	**Sistema**
	1		*Mostra M1*
	2	*Clicca sul pulsante* `Cerca`	
	3		*Mostra M2*
	4	*Clicca sul pulsante* `Filtra`	
	5		*Mostra M3*
	6	*Seleziona il tipo di filtro*	
	7		*Mostra M4*
	8	*Clicca sul pulsante* `Apply`	
	9		*Mostra M5*

3.1.2 Prototipazione visuale via Mock-up

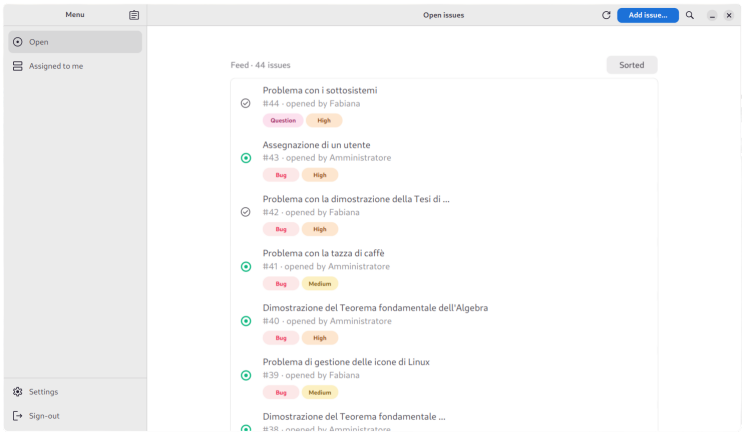


Figura 3.1: Mock-up *M1*

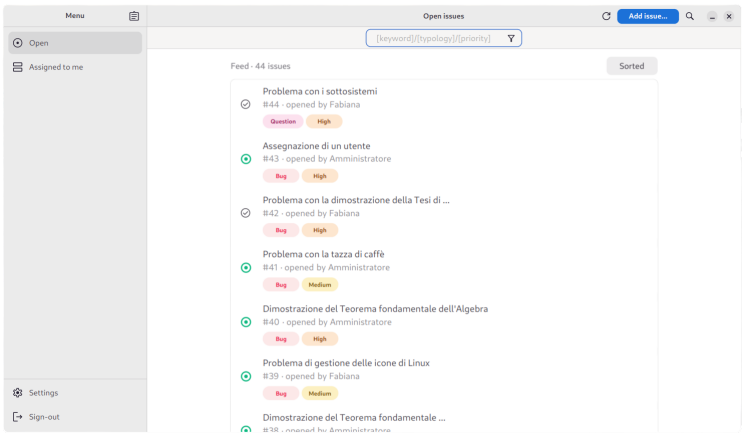


Figura 3.2: Mock-up *M2*

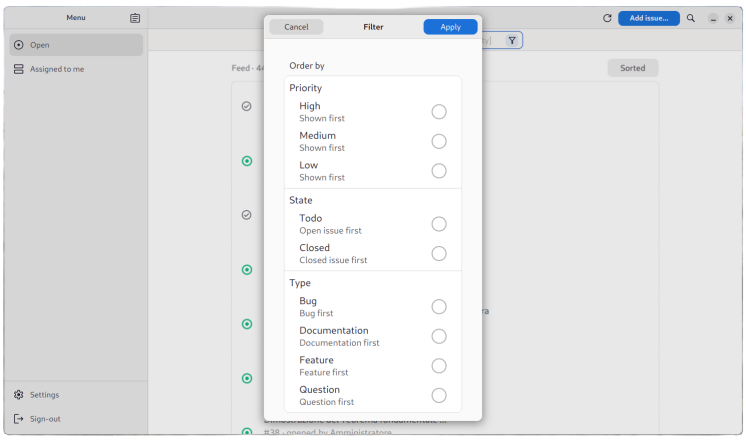


Figura 3.3: Mock-up *M3*

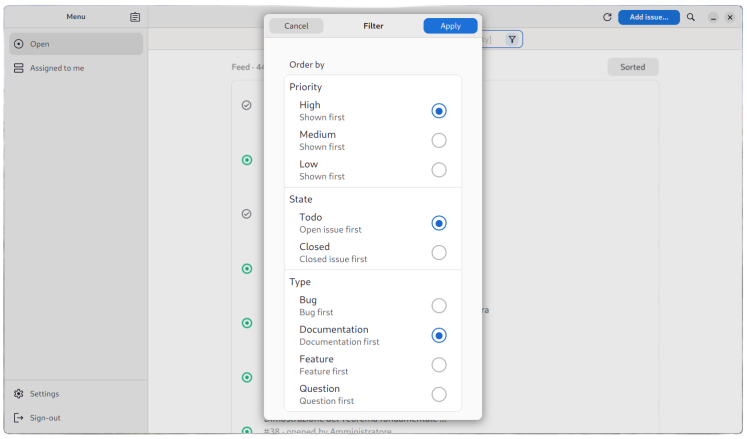


Figura 3.4: Mock-up *M4*

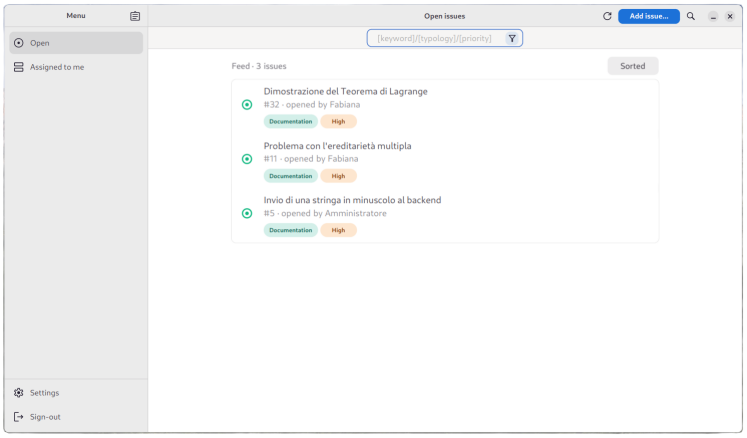


Figura 3.5: Mock-up *M4*

3.2 Requisiti non funzionali

3.2.1 Requisiti del prodotto

RP1 (*Requisito di affidabilità*): Il sistema deve offrire un alto grado di probabilità con cui fornisca correttamente i servizi come atteso dal cliente in un determinato periodo di tempo.

RP2 (*Requisito di apprendimento*): Il sistema deve offrire un alto grado di *familiarizzazione* dell'utente con le funzioni del prodotto entro un determinato periodo di tempo.

RP3 (*Requisito di adeguatezza*): Il sistema deve facilitare il raggiungimento dei compiti e degli obiettivi da parte dell'utente.

RP4 (*Requisito di portabilità*): Una parte del sistema (*back-end*) può essere eseguita in ambienti diversi. Il termine ambiente può riferirsi alla piattaforma hardware o all'ambiente software.

RP5 (*Requisito di confidenzialità*): Il sistema deve impedire accessi non autorizzati a informazioni riservate.

3.2.2 Requisiti organizzativi

RO1 (*Requisito sull'architettura*): Il sistema deve basarsi su una architettura distribuita.
Requisiti di sistema:

- Il sistema deve prevederre due macro-componenti indipendenti: *back-end* e *front-end*.
- Il *back-end* deve essere composto da almeno una componente che gestisce la logica di business e i dati.
- Il *back-end* deve offire una interfaccia di programmazione, con cui comunicare, mediante *API REST*.
- Il *front-end* deve comunicare con il *back-end* solo tramite le *API* esposte.
- Il *front-end* deve essere realizzato come applicazione desktop, come applicazione web, oppure come applicazione mobile.

RO2 (*Requisito di implementazione*): Per la fase di implementazione devono essere utilizzati linguaggi di programmazione Object-Oriented.
Requisiti di sistema:

- Sia il *back-end* che il *front-end* devono essere realizzati utilizzando linguaggi del paradigma ad oggetti.
- Il *front-end* dovrebbe essere un'applicazione desktop realizzata con Java e le librerie grafiche Swing o JavaFX.

RO3 (*Requisito sugli strumenti*): In fase di produzione devono essere utilizzati tool di CASE.
Requisiti di sistema:

- Strumenti come Visual Paradigm o Modelio devono essere utilizzati in fase di progettazione.

RO4 (*Requisito di consegna*): Il prodotto software e la relativa documentazione devono essere consegnati entro il 30 Settembre 2026.
Requisiti di sistema:

- Per il dettaglio completo delle scadenze di progetto si rimanda al capitolo **8** del documento **Progetto di Ingegneria del software.pdf**.

RO5 (*Requisito di unicità*): Il sistema deve essere progettato rispettando i requisiti antiplagio, in grado di soddisfare e superare gli standard richiesti in materia di originalità dei contenuti.
Requisiti di sistema:

- Tutti gli artefatti consegnati vengono processati con software antiplagio Turnitin, e raffrontati automaticamente con progetti degli anni scorsi e di quest'anno.

Capitolo 4

Design del sistema

Il primo modello del sistema che si è definito (*modello di analisi*) è stato il risultato della prima fase, ovvero scaturito dalla fase di *analisi e definizione dei requisiti*.

L'obiettivo di questo capitolo è invece descrivere il nuovo modello di sistema (*modello di progettazione*), focalizzando l'attenzione prima sull'architettura scelta, poi sulla scomposizione del sistema in sottosistemi e infine sulla descrizione degli obiettivi di design raggiunti.

4.1 Modello architetturale client-server

Lo stile architetturale *client-server* è stato organizzato in tre livelli (*tier*) dove ognuno di questi corrisponde a un nodo fisico di calcolo su cui è distribuito il sistema.

Livello	**Realizzazione**	**Responsabilità**
Presentazione (*client tier*)	Applicazione desktop JavaFX	Mostrare i dati e raccogliere le azioni dell'utente
Logica di business (*application tier*)	Back-end Spring Boot	Validare i dati, applicare le regole di business, autenticare gli utenti e controllarne i permessi
Dati (*database tier*)	Database MySQL	Memorizzare i dati in modo permanente e mantenerli coerenti

4.1.1 Livello di presentazione (*client-tier*)

Il livello di presentazione costituisce l'interfaccia utente (UI) del sistema e include la definizione dei layout visuali, delle schermate e della User Experience (UX) complessiva. Per lo sviluppo di questa applicazione desktop si è optato per l'impiego della libreria JavaFX.

Questo layer assolve a due funzioni primarie:

1 (*Visualizzazione dei dati*): eroga a schermo le informazioni precedentemente elaborate dal server.

2 (*Acquisizione degli input*): intercetta le interazioni dell'utente (eventi di puntamento, digitazione di testo, selezioni da menu) e le inoltra ai livelli sottostanti.

Il funzionamento si articola su un modello lineare: il livello riceve la richiesta dell'utente, la delega al layer della logica applicativa per la relativa elaborazione e, infine, renderizza il risultato ottenuto.

In linea con i principi di separazione delle responsabilità, questo livello non implementa alcuna logica decisionale: non valuta la validità o l'ammissibilità delle operazioni richieste, ma si limita a demandare tali verifiche al livello sottostante e a rappresentare graficamente l'esito della risposta ricevuta.

4.1.2 Livello della logica business (*application-tier*)

Il livello della logica applicativa rappresenta il modulo centrale del sistema: in esso risiedono i processi decisionali e l'esecuzione delle operazioni fondamentali per il funzionamento dell'applicazione.

Per lo sviluppo di questo layer è stato impiegato il linguaggio Java unitamente al framework Spring. Il layer risponde all'architettura sottostante esponendo le proprie funzionalità al livello di presentazione tramite API REST con scambio dati in formato JSON. Contestualmente, si interfaccia con il livello dei dati (Data Access Layer) per le operazioni di persistenza, lettura e aggiornamento delle informazioni.

4.1.3 Livello di gestione dei dati (*database-tier*)

Il livello di persistenza è responsabile della memorizzazione a lungo termine, del recupero e della salvaguardia della coerenza delle informazioni.

La coerenza è garantita dai vincoli dello schema, come le chiavi esterne, che impediscono per esempio di avere un'issue senza autore. Al database accede solo il back-end: il client non comunica mai direttamente con esso.

4.1.4 Perché conviene separare i livelli

I vantaggi di un'architettura a livelli sono:

1 (*Manutenibilità*): Ogni componente ha una singola responsabilità. Trovare e correggere gli errori diventa più rapido e intuitivo.

2 (*Flessibilità*): I moduli sono indipendenti. È possibile rinnovare l'interfaccia (es. da desktop a web) o cambiare il database senza impattare sul resto del sistema.

3 (*Sicurezza*): Il database rimane isolato dagli accessi esterni diretti, mentre l'autenticazione e il controllo degli accessi sono gestiti in un unico punto sul server.

4 (*Scalabilità*): I livelli possono essere eseguiti su macchine diverse, come descritto nel paragrafo riguardante il deployment.

4.2 Decomposizione del sistema

Il sistema software in esame è stato progettato adottando un approccio modulare, partendo da componenti elementari più facilmente gestibili in fase di sviluppo e manutenzione.

In coerenza con l'architettura scelta, il sistema si articola in due macro-sottosistemi: *Front-end* e *Back-end*. Lo sviluppo del primo è stato affidato al team guidato da Giandomenico Iameo, mentre il secondo è stato coordinato dal team di Ubaldo Musto.
Ciascun sottosistema è stato a sua volta suddiviso in moduli minori, seguendo il principio di separazione delle responsabilità.

Nel seguente paragrafo, verrano descritti dapprima i pattern architetturali utilizzati per la strategia di comunicazione tra i diversi componenti del sottosistema, per poi mostrare la stratificazione del sottosistema stesso.

4.2.1 Front-end

Il *Front-end* provvede alla rappresentazione dei dati e a tutte le interazioni con l'utente.

La soluzione adottata per suddividere i moduli presenti all'interno di questo sottosistema si basa sul pattern **MVP-Passive** *(Model-View-Presenter)*. Per una trattazione approfondita dei meccanismi di funzionamento e delle specifiche di questo pattern, si rimanda all'apposito paragrafo dedicato.

Il diagramma successivo mostra la stratificazione del front-end e la collaborazione tra gli strati stessi.

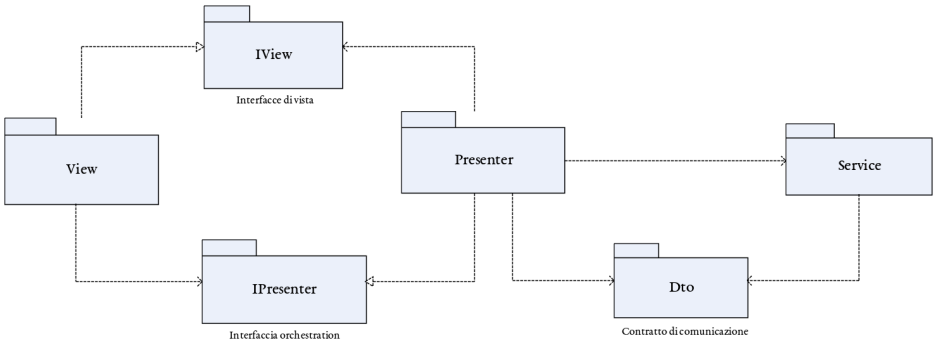


Figura 4.1: Diagramma dei package del front-end

I componenti grafici e il layout dell'applicazione si trovano nello strato *View*. Non contengono logica sui dati, ma si limitano a ricevere i valori già pronti dallo strato *Presenter* e ne invocano i contratti t ramite le interfacce di `IPresenter`.

Sebbene la logica applicativa sia definita altrove, i componenti grafici mantengono comunque una natura reattiva rispetto alle interazioni dell'utente: a questo scopo, sono state configurate le azioni in risposta agli eventi generati dall'utente.

Il *Presenter* rappresenta lo strato di orchestrazione. Il funzionamento di questo strato si basa su un modello lineare: Il *Presenter* intercetta i dati provenienti dalle interfacce di vista, converte tali dati in conformità con il contratto di comunicazione definito con il back-end e inoltra la richiesta. Successivamente riceve l'esito della chiamata dal server e, avvalendosi di un componente di mapping (mapper), adatta i dati ricevuti per restituirli alla *View* pronti per la visualizzazione.

Rispetto al classico modello MVP, il front-end non integra un *modello di dominio* locale, ma si affida a uno specifico livello *Service* che comunica direttamente con il *Model* del back-end. Il modello espresso dal front-end è più precisamente un *modello di presentazione*.

Più nel dettaglio, a livello di vista è stato adottato il *pattern Composite* per organizzare i componenti grafici in una struttura ad albero. La motivazione che ha spinto a scegliere tale pattern risiede nella natura del toolkit grafico di JavaFX: la cui architettura si fonda nativamente su un grafo di nodi.

Il DTO, invece, rappresenta il *modello dei dati* e, insieme all'albero dei componenti grafici, rappresenta l'unico altro modello definito all'interno del sottosistema.

La ragione che ha indotto a non definire un *modello di dominio* locale è di natura progettuale: Il front-end è un client "leggero": riceve i dati dal server in formato DTO, li traduce e li mostra. Un livello *domain* avrebbe duplicato quello del server senza aggiungere regole.

4.2.1.1 Pattern MVP-*Passive*

MVP-Passive è un modello architetturale comunemente utilizzato nello sviluppo di applicazioni mobili e desktop. Si tratta di un'evoluzione del modello MVC (*Model-View-Controller*), incentrato su una separazione delle responsabilità che garantisce maggiore flessibilità e scalabilità.

Il modello MVP è noto principalmente per semplificare la manutenibilità dell'applicazione, favorendone il miglioramento e la scalabilità nel tempo.

Le proprietà principali di questo modello sono la rappresentazione dei dati e la definizione della logica di business dell'applicazione. È responsabile del recupero, dell'archiviazione e della manipolazione dei dati, nonché dell'applicazione delle regole aziendali. Il modello è indipendente dall'interfaccia utente e dovrebbe essere indipendente dall'implementazione specifica della vista (*View*) e del presentatore (*Presenter*).

Il pattern si articola in tre componenti principali, come si nota dalla Figura 1.2.

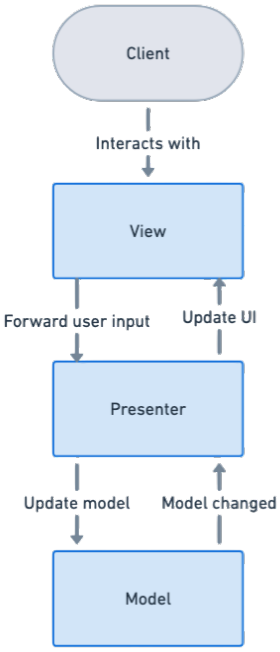


Figura 4.2: Pattern MVP-passive

1 (*View*): È responsabile della presentazione dei dati e dell'acquisizione dell'input dell'utente. È un *componente passivo* che si basa sul *Presenter* per aggiornare il proprio stato e gestire le interazioni dell'utente.
La vista in genere è costituita da elementi visivi come i componenti grafici dell'interfaccia utente (UI) e il layout dell'applicazione. L'obiettivo principale di View è garantire che l'interfaccia utente sia reattiva, visivamente accattivante e facile da navigare.

2 (*Presenter*): Il *Presenter* funge da mediatore tra *Model* e *View*, orchestrando il flusso di dati e gestendo le interazioni dell'utente. È responsabile del recupero dei dati dal *Model*, della loro elaborazione e dell'aggiornamento della *View* di conseguenza.
Il *Presenter* gestisce inoltre gli eventi utente dalla *View*, come i clic sui pulsanti o le modifiche di input, attivaando azioni specifiche in base a questi eventi.
Separando gli aspetti della gestione dei dati (*Model*), dell'interfaccia utente (*View*) e della componente di presentazione (*Presenter*), il pattern consente lo sviluppo di applicazioni più modulari, scalabili e manutenibili.

3 (*Model*): rappresenta i dati e la logica aziendale dell'applicazione, simile al modello in MVC. È responsabile dell' elaborazione, archiviazione e gestione dei dati e dell'implementazione di tutte le regole aziendali necessarie. Il modello non comunica direttamente con la vista.

Concludendo questo paragrafo sugli obiettivi che permette di raggiungere questo pattern, si possono certamente menzionare la separazione delle responsabilità tra i componenti, la manutenibilità e la scalabilità dell'applicazione; proprietà fondamentali che consentono di soddisfare le richieste in continua evoluzione da parte dei clienti.

Come esemplificato da molte piattaforme, il modello può essere integrato perfettamente in vari strumenti e metodologie di sviluppo, consolidando ulteriormente la sua rilevanza e importanza nel mondo dello sviluppo di applicazioni.

Nel progetto i ruoli del pattern sono realizzati come segue. La *View* comprende le schermate e i componenti grafici descritti dai file FXML; il *Presenter* comprende gli orchestrator e i loro collaboratori (assembler e mapper); il *Model* risiede nel back-end, che il front-end raggiunge attraverso il livello `Service` e il package `dto`.

Rispetto alla formulazione classica, il *Presenter* è articolato in più componenti ed è associato alle funzionalità anziché alle singole schermate; *View* e *Presenter* si conoscono soltanto tramite le interfacce dei livelli *IView* e *IPresenter*.

4.2.1.2 Stratificazione del sottosistema

Il sottosistema front-end è stato ulteriormente decomposto in parti più piccole in base alle responsabilità che ricopre; come si evince dalla Figura 3.5 riportata qui di seguito.

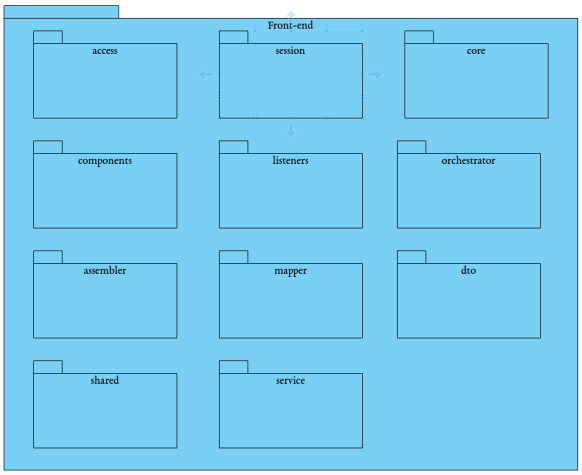


Figura 4.3: Front-end

4.2.2 Back-end: pattern Controller-Service-Repository

Il back-end è organizzato in livelli secondo il pattern Controller-Service-Repository. Lo abbiamo scelto perché separa in modo netto le responsabilità: i controller gestiscono la comunicazione HTTP con il client, i service contengono tutte le regole di business e i repository l'accesso al database.

In questo modo la logica si trova in un unico punto e può essere riusata da più

endpoint, ogni livello può cambiare senza toccare gli altri e i service possono essere testati in isolamento, sostituendo i repository con dei mock.

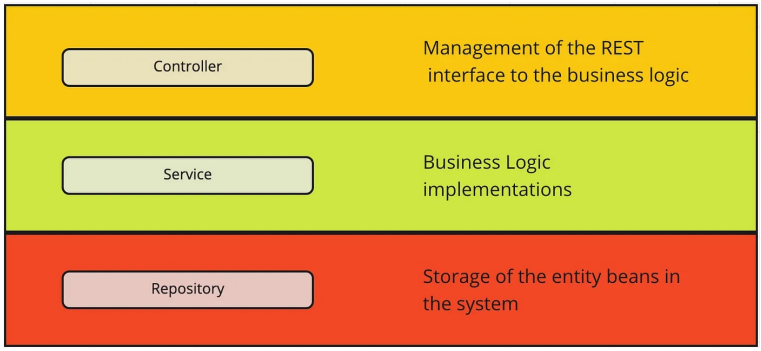


Figura 4.4: Pattern Controller-Service-Repository

- *Controller*: In cima allo schema, ha il solo compito di esporre le funzionalità del sistema come API REST, in modo che possano essere usate da entità esterne, come il client.
- *Service*: Contiene tutta la logica di business. Se ha bisogno di leggere o salvare dati, si rivolge a un repository; chi vuole accedere alla logica di business deve passare da un controller.
- *Repository*: In fondo allo schema, ha il compito di salvare e recuperare un certo insieme di dati.

4.2.3 Back-end: Repository pattern

Per l'accesso ai dati abbiamo adottato il Repository pattern, che offre ai service un modo astratto di lavorare con i dati. Ogni entità del dominio ha un proprio repository, che si comporta come una collezione di oggetti: i service possono cercare, salvare ed eliminare elementi senza sapere come sono memorizzati nel database.

I repository sono interfacce: le operazioni CRUD di base (creazione, lettura, aggiornamento ed eliminazione) sono ereditate da Spring Data JPA, che ne genera l'implementazione, mentre le ricerche con filtri e ordinamenti sono query aggiuntive dichiarate nell'interfaccia stessa. In questo modo i service non contengono query né dettagli delle tabelle: un cambiamento nello schema si riflette sui repository e non sui service.

4.3 Comunicazione tra front-end e back-end

Il client e il server comunicano tramite API REST su HTTP. Ogni operazione dell'utente corrisponde a una richiesta HTTP, con dati in formato JSON sia in richiesta sia in risposta; fanno eccezione le immagini allegate alle issue, che vengono scaricate con una richiesta separata come dati binari, per non appesantire gli elenchi.

Gli errori hanno sempre lo stesso formato: un codice HTTP (per esempio 400 per

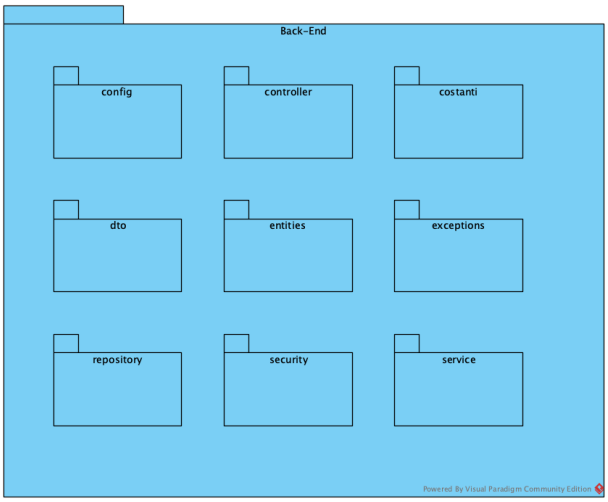


Figura 4.5: Decomposizione del back-end

dati non validi, 403 per permessi insufficienti, 404 per una risorsa inesistente) e un corpo JSON con un messaggio che il client può mostrare all'utente.

4.3.1 Autenticazione

L'autenticazione usa Spring Security con la sessione HTTP: al login il server verifica email e password, crea una sessione e restituisce al client un cookie, che il client allega a tutte le richieste successive. Le password sono salvate solo come hash BCrypt.

4.3.2 Interfaccia delle API

La tabella seguente riassume gli endpoint esposti dal back-end. Tutti, tranne il login, richiedono l'autenticazione.

Metodo	Endpoint	Ruolo	Funzionalità
POST	`/api/auth/login`	pubblico	Login
GET	`/api/auth/me`	autenticato	Utente collegato
POST	`/api/auth/logout`	autenticato	Logout
POST	`/api/utenti`	amministratore	Crea utente
GET	`/api/utenti`	amministratore	Elenco utenti
GET	`/api/utenti/suggeriti`	amministratore	Suggerimento assegnatari
GET	`/api/issues`	autenticato	Elenco con filtri e ordinamento
GET	`/api/issues/ricerca`	autenticato	Ricerca per parola chiave
GET	`/api/issues/{id}`	autenticato	Dettaglio issue
GET	`/api/issues/{id}/immagine`	autenticato	Immagine allegata
POST	`/api/issues`	amm., normale	Crea issue
POST	`/api/issues/{id}/chiudi`	amministratore	Chiudi issue
POST	`/api/bugs/{id}/assegna`	amministratore	Assegna bug
POST	`/api/bugs/{id}/duplicato`	amministratore	Segna come duplicato
GET	`/api/bugs/assegnati`	normale	Bug assegnati
GET	`/api/bugs/assegnati/aperti`	normale	Bug assegnati aperti
GET	`/api/notifiche`	normale	Tutte le notifiche
GET	`/api/notifiche/conteggio`	normale	Numero di notifiche nuove
POST	`/api/notifiche/leggi-nuove`	normale	Legge le notifiche nuove

4.4 Deployment

Il diagramma di deployment mostra come i componenti software sono distribuiti sui nodi di esecuzione.

- *Macchina dell'utente*: esegue il client JavaFX, che richiede un ambiente Java 21.

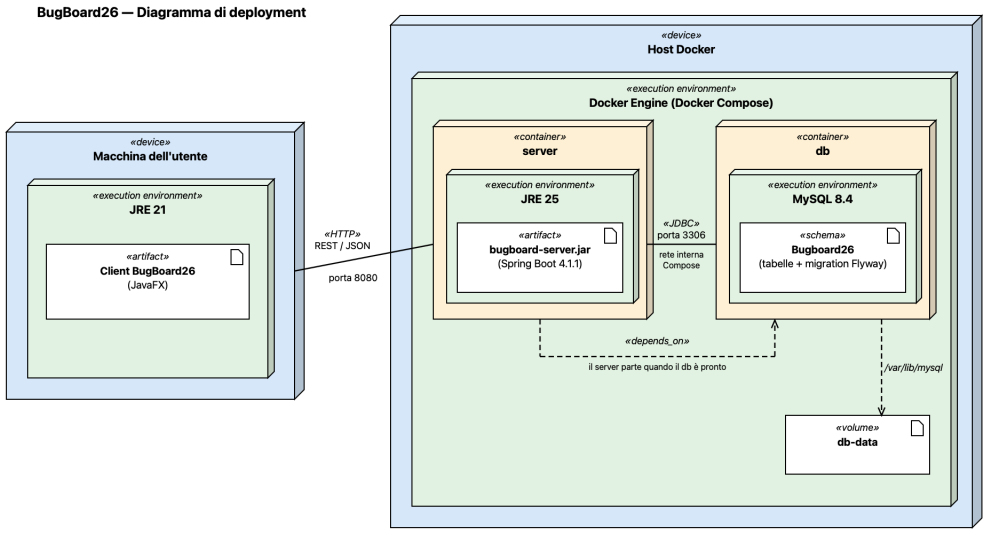


Figura 4.6: Diagramma di deployment

- *Host Docker*: esegue due container avviati insieme da Docker Compose. Il container *server* contiene il back-end Spring Boot con il runtime Java 25 ed è raggiungibile sulla porta 8080; il container *db* contiene MySQL 8.4 e salva i dati in un volume, così non si perdono quando il container viene ricreato.
- *Comunicazioni*: il client comunica con il back-end tramite HTTP, scambiando dati in formato JSON; il back-end comunica con il database tramite JDBC, sulla rete interna creata da Docker Compose.

Client e back-end si trovano quindi sempre su nodi distinti. Back-end e database sono in container separati e, nella configurazione consegnata, sullo stesso host; per spostarli su macchine diverse basta cambiare l'indirizzo del database nella configurazione del back-end.

4.5 Criteri di design

- *Separazione delle responsabilità*: ogni livello e ogni componente ha un solo compito. Il principio è applicato a tutti i livelli: nell'architettura a tre livelli, nel pattern Controller-Service-Repository del back-end e nel pattern MVP del front-end.
- *Indipendenza tra front-end e back-end*: le due parti comunicano solo tramite API REST, secondo un contratto definito dai DTO. Il back-end non conosce il client, che può essere sostituito senza modifiche al server.
- *Stato e regole centralizzati*: dati, regole di business e controlli dei permessi stanno solo nel back-end. Il client mostra i dati e inoltra le richieste, ma non prende decisioni.

- *Astrazione per il riuso e l'estensione*: i ruoli degli utenti e i tipi di issue sono gerarchie di classi, l'accesso ai dati passa per interfacce *repository* e i componenti grafici seguono il pattern *Composite*.
- *Sicurezza per default*: ogni operazione, tranne il login, richiede l'autenticazione, e i permessi sono dichiarati su ogni endpoint.
- *Testabilità*: i service dipendono dai repository solo tramite interfacce, quindi possono essere testati in isolamento sostituendo i repository con dei mock.

4.6 Scelte tecnologiche

Ambito	**Tecnologia**
Front-end	Java, JavaFX
Back-end	Java, Spring Boot (Web, Data JPA, Security, Validation)
Database	MySQL
Gestione dello schema	Flyway
Distribuzione	Docker, Docker Compose

4.6.1 Spring Boot

Per il back-end abbiamo scelto Spring Boot, il framework più diffuso per lo sviluppo di servizi REST in Java. Spring Boot segue il paradigma *convention over configuration*: fornisce una configurazione predefinita ragionevole per gli aspetti più comuni (server web integrato, connessione al database, conversione JSON), così gli sviluppatori devono specificare solo ciò che si discosta dallo standard. Questo riduce il codice di configurazione e il numero di decisioni da prendere, senza perdere flessibilità: ogni impostazione predefinita può essere sovrascritta quando serve.

Spring offre inoltre la *dependency injection*: controller, service e repository vengono creati dal framework e collegati tra loro automaticamente, il che rende i componenti indipendenti e facili da testare. Infine il suo ecosistema mette a disposizione moduli già integrati per tutte le esigenze del progetto: Spring Web per le API REST, Spring Data JPA per l'accesso al database, Spring Security per l'autenticazione e i permessi, Jakarta Validation per il controllo dei dati in ingresso. Usare soluzioni standard e collaudate, invece di scriverle da zero, ci ha permesso di concentrarci sulla logica di business, riducendo gli errori soprattutto in un ambito delicato come la sicurezza.

4.6.2 MySQL e Flyway

Per la persistenza abbiamo usato MySQL, un database relazionale. I dati di BugBoard26 sono strutturati e collegati tra loro (un'issue ha un autore, un bug ha un assegnatario,

una notifica ha un destinatario), quindi il modello relazionale, con le sue chiavi esterne e le transazioni, è il più adatto.

Lo schema non viene generato dal codice, ma è creato e aggiornato da Flyway tramite script SQL numerati (*migration*), applicati automaticamente all'avvio del back-end. In questo modo ogni installazione, dal computer di sviluppo al container, ha esattamente lo stesso schema.

4.6.3 Docker

Docker è usato per automatizzare la distribuzione del back-end sotto forma di container, cioè un pacchetto che contiene l'applicazione insieme a tutto ciò che le serve per funzionare (il runtime Java e le librerie). Il container si comporta allo stesso modo su qualsiasi macchina, dal computer di sviluppo a un server nel cloud, ed elimina i problemi di installazione e configurazione dell'ambiente.

Il Dockerfile è di tipo *multi-stage*: l'applicazione viene compilata in un'immagine con il JDK e Maven, mentre l'immagine finale contiene solo il runtime Java e l'applicazione, quindi è più leggera e non include strumenti di compilazione. Infine, con Docker Compose il back-end e il database MySQL si avviano insieme con un unico comando: il back-end parte solo quando il database è pronto, e i dati vengono conservati in un volume, così non si perdono quando i container vengono riavviati.

4.6.4 JavaFX

Per questa applicazione desktop è stato utilizzato JavaFX come toolkit grafico. Tra i suoi vantaggi merita di essere menzionata la gestione del layout, che permette di separarlo dal codice. Il layout si trova in file FXML, che descrivono in modo dichiarativo i componenti grafici dell'applicazione. Ogni file è associato a un controller, cioè a una classe che ne definisce il comportamento in risposta alle interazioni dell'utente: senza controller l'interfaccia è priva di comportamento.

L'aspetto grafico è definito in fogli di stile CSS e si applica ai componenti tramite classi di stile, senza modificare il codice. Infine, lo stesso codice Java può essere eseguito su più piattaforme, come Windows, macOS e Linux.

4.6.5 Linguaggi e strumenti

Il sistema è stato sviluppato con un linguaggio orientato agli oggetti, Java, usato sia per il back-end sia per il front-end. Usare lo stesso linguaggio nei due componenti ha semplificato lo sviluppo e la collaborazione tra i membri del gruppo.

- *Java*: versione 25 per il back-end, con il framework Spring Boot, e versione 21 per il front-end, con la libreria JavaFX.
- *SQL*: usato per definire lo schema del database e i dati iniziali, negli script di migrazione gestiti da Flyway.

- *JPQL* (*Java Persistence Query Language*): linguaggio di interrogazione simile a SQL, che lavora sulle classi Java invece che sulle tabelle. È usato nei repository per le ricerche con filtri e ordinamenti.
- *YAML*: formato del file di configurazione del back-end e del file di Docker Compose.

Gli strumenti utilizzati sono:

- *IntelliJ IDEA*: ambiente di sviluppo per entrambi i sottosistemi;
- *Maven*: gestione delle dipendenze e build automatica del back-end;
- *Visual Paradigm* e *Modelio*: strumenti CASE per i diagrammi UML, rispettivamente del back-end e dei casi d'uso, e del front-end;
- *MySQL Workbench*: progettazione del modello ER e generazione automatica dello script SQL di creazione delle tabelle;
- *Git* e *GitHub*: controllo di versione e hosting dei repository del front-end e del back-end;
- *SonarQube*: analisi statica della qualità del codice del back-end.

Capitolo 5

Verifica e convalida

5.1 Progettazione dei casi di test

Questa sezione descrive nel dettaglio la progettazione dei casi di test e le motivazioni delle strategie adottate. L'obiettivo è definire un insieme di casi di test efficace, in grado di verificare che il sistema soddisfi i requisiti.

Tra i diversi approcci alla progettazione dei test, la strategia adottata si basa sul partizionamento in classi di equivalenza. Il dominio dei dati in ingresso viene suddiviso in classi, ciascuna composta da valori per cui il sistema dovrebbe comportarsi in modo equivalente, e per ogni classe si sceglie almeno un caso di test rappresentativo. Questa scelta riduce il numero di test necessari senza rinunciare alla copertura dei comportamenti distinti del sistema.

Il test plan completa i test di unità: verifica la funzionalità nel suo insieme, con il back- end avviato e collegato al database, e controlla anche gli aspetti che i test di unità non vedono, cioè i permessi, la validazione delle richieste e i codici di risposta.

5.2 Descrizione dei casi di testi in JUnit

La presente sezione si occuperà di rappresentare i casi di test definiti per esaminare il comportamento delle funzionalità più complesse e per accertarne il corretto funzionamento. A questo scopo, è stato utilizzato il framework JUnit 5.

Per facilitare la leggibilità, i casi di test sono stati suddivisi in base al sottosistema di appartenenza. I primi casi di test che verranno illustrati sono quelli definiti nella parte front-end; successivamente verranno descritti quelli della parte back-end.

I metodi selezionati (lato front-end) per gli Unit Test sono i seguenti:

1. `selectToggleFilter()`: imposta un filtro di ricerca a partire dai campi impostati nell'interfaccia grafica. Lo scopo è verificare che ogni combinazione di scelte, compresa l'assenza di selezione, produca un filtro valido.

2 `setDate()`: traduce una data in formato UTC in una data leggibile dall'utente. Lo scopo è verificare che le date significative (passaggio di mezzanotte o di fine anno, valore mancante o malformato) siano convertite correttamente.

Per la definizione delle classi di equivalenza è stato adottato l'*approccio basato sulla funzionalità*, in cui il processo di partizionamento si basa rigorosamente sui requisiti e non sull'implementazione del codice.

Le classi di equivalenza sono pensate in relazione al comportamento del metodo. Quindi i parametri non sono stati valutati in modo isolato, ma si è tenuto conto del contesto e delle relazioni logiche esistenti tra di essi.

5.2.1 Metodo `selectToggleFilter()`

La firma del metodo è la seguente:

```
void selectToggleFilter(ToggleGroup group, String filterValue)
```

`filterValue` rappresenta la tipologia di filtro selezionata appartenente ad uno specifico gruppo, il quale è stato indicato nel primo parametro, `group`.

Per il parametro `group` il dominio è $G = \{\text{typologyGroup}, \text{stateGroup}, \text{priorityGroup}\}$ I gruppi vengono creati nativamente dall'interfaccia tramite FXML, venendo già popolati con i rispettivi `RadioButton`. Inoltre, il riferimento non è mai nullo e si ottiene la seguente partizione

$$C_1 = \{\text{typologyGroup}\}, C_2 = \{\text{stateGroup}\}, C_3 = \{\text{priorityGroup}\}$$

Per il parametro `filterValue`, sia $S = \{w | w \text{ rappresenta una stringa}\} \cup \{NIL\}$ l'insieme di tutte le stringhe incluso il valore nullo, sono state identificate le seguenti classi di equivalenza

$$C_4 = \{\text{BUG}, \text{FEATURE}, \text{DOCUMENTATION}, \text{QUESTION}\}$$

$$C_5 = \{\text{TODO}, \text{CHIUSO}\}, C_6 = \{\text{ALTA}, \text{MEDIA}, \text{BASSA}\}$$

$$C_7 = S \setminus \bigcup_{i=4}^{6} C_i$$

Secondo l'implementazione del metodo `selectToggleFilter`, ogni classe è costituita da valori validi. Per quanto riguarda la classe C_7, se la stringa non corrisponde a nessun `UserData` dei `RadioButton` definiti, il metodo lascia il gruppo senza selezione.

Il criterio di copertura adottato è *Each Choice Coverage*, che ha permesso di generare la seguente batteria di test

```
@Test
    void testFilterTypologyValid()  {
        FilterFake view = new FilterFake();
        ToggleGroup typologyGroup = new ToggleGroup();

        view.toggle("BUG",
                typologyGroup);
        view.toggle("FEATURE",
                typologyGroup);
        view.toggle("DOCUMENTATION",
                typologyGroup);
        view.toggle("QUESTION",
                typologyGroup);

        view.selectToggleGroup(typologyGroup,
                "BUG");

        assertNotNull(typologyGroup
                .getSelectedToggle());
        assertEquals("BUG", typologyGroup
                .getSelectedToggle()
                .getUserData());
    }

    @Test
    void testFilterPriorityValid()  {
        FilterFake view = new FilterFake();
        ToggleGroup priorityGroup = new ToggleGroup();

        view.toggle("ALTA",
                priorityGroup);
        view.toggle("MEDIA",
                priorityGroup);
        view.toggle("BASSA",
                priorityGroup);

        view.selectToggleGroup(priorityGroup,
                "ALTA");

        assertNotNull(priorityGroup
                .getSelectedToggle());
        assertEquals("ALTA", priorityGroup
                .getSelectedToggle()
                .getUserData());
    }
```

```
@Test
void testFilterStateValid()  {
    FilterFake view = new FilterFake();
    ToggleGroup stateGroup = new ToggleGroup();

    view.toggle("TODO",
            stateGroup);
    view.toggle("CHIUSO",
            stateGroup);

    view.selectToggleGroup(stateGroup,
            "TODO");

    assertNotNull(stateGroup
            .getSelectedToggle());
    assertEquals("TODO", stateGroup
            .getSelectedToggle()
            .getUserData());
}

@Test
void testFilterBadValid()  {
    FilterFake view = new FilterFake();
    ToggleGroup stateGroup = new ToggleGroup();

    view.toggle("TODO",
            stateGroup);
    view.toggle("CHIUSO",
            stateGroup);

    view.selectToggleGroup(stateGroup,
            "APERTO");
    assertNull(stateGroup
            .getSelectedToggle());
}
```

Tabella 5.1: Casi di test

Caso	`group`	`filterValue`	Classi	Esito atteso
TC1	`typologyGroup`	`BUG`	C_1, C_4	pulsante BUG selezionato
TC2	`priorityGroup`	`ALTA`	C_3, C_6	pulsante ALTA selezionato
TC3	`stateGroup`	`TODO`	C_2, C_5	pulsante TODO selezionato
TC4	`stateGroup`	`BLOCCATO`	C_2, C_7	nessuna selezione

5.2.2 Metodo `setDate()`

In assenza di metodi pubblici non banali con almeno due parametri indipendenti dal back-end, la scelta è ricaduta su `setDate()`: pur accettando un solo parametro, è uno dei metodi più complessi ed è condiviso da più componenti, anziché appartenere a una specifica funzionalità.

La firma del metodo è la seguente:

```
void setDate(LocalDateTime date)
```

Il metodo presenta un dominio di input molto ampio. In base al comportamento atteso, in cui ad esempio intervalli inferiori a un minuto vengono tutti indicati come "*Just now*", è stata ricavata la seguente partizione:

$$C_1 = \{NIL\}, C_2 = \{w \mid w \text{ rappresenta una data del futuro}\},$$

$$C_3 = \{\Delta \mid 0 \leq \Delta < 60\text{s}\}$$

dove Δ rappresenta il tempo trascorso tra la data fornita e il tempo attuale.

$$C_4 = \{\Delta \mid 0 < \Delta < 59\text{min}\}, C_5 = \{\Delta \mid 0 < \Delta < 24\text{h}\}, C_6 = \{\Delta \mid 0 < \Delta < 7\text{d}\}$$

$$C_7 = \{\Delta \mid 7d \leq \Delta < 1\text{m}\}, C_8 = \{\Delta \mid 0 < \Delta < 12\text{m}\},$$

$$C_9 = \{\Delta \mid \Delta > 0\}$$

dove 1m rappresenta 1 mese, 7d rappresenta 7 giorni e l'ultima classe di equivalenza esprime l'intervallo in anni.

Il criterio di copertura adottato è *Each Choice Coverage*, che ha permesso di generare la seguente batteria di test.

```
@Test
void testDateNull()  {
    assertThrows(NullPointerException.class,
            () -> {
        NoticeDate notice =
                new NoticeDate();
        notice.setDate(null);
    });
}

@Test
void testPastDateYear()  {
    NoticeDate view = new NoticeDate();
    view.setDate(LocalDateTime
            .now(ZoneOffset.UTC)
            .minusYears(3));

    assertNotNull(view);
    assertEquals("3 years ago", view
            .getDate());
}

@Test
void testPastDateMonth()  {
    NoticeDate view = new NoticeDate();
    view.setDate(LocalDateTime
            .now(ZoneOffset.UTC)
            .minusMonths(11));

    assertNotNull(view);
    assertEquals("11 months ago", view
            .getDate());
}

@Test
void testPastDateWeek()  {
    NoticeDate view = new NoticeDate();
    view.setDate(LocalDateTime
            .now(ZoneOffset.UTC)
            .minusDays(14));

    assertNotNull(view);
    assertEquals("2 weeks ago", view
            .getDate());
}
```

```
@Test
void testPastDateDay()  {
    NoticeDate view = new NoticeDate();
    view.setDate(LocalDateTime
            .now(ZoneOffset.UTC)
            .minusDays(6));

    assertNotNull(view);
    assertEquals("6 days ago", view
            .getDate());
}

@Test
void testPastDateHour()  {
    NoticeDate view = new NoticeDate();
    view.setDate(LocalDateTime
            .now(ZoneOffset.UTC)
            .minusHours(23));

    assertNotNull(view);
    assertEquals("23 hours ago", view
            .getDate());
}

@Test
void testPastDateMinute()  {
    NoticeDate view = new NoticeDate();
    view.setDate(LocalDateTime
            .now(ZoneOffset.UTC)
            .minusMinutes(59));

    assertNotNull(view);
    assertEquals("59 minutes ago", view
            .getDate());
}

@Test
void testPastDateSecond()  {
    NoticeDate view = new NoticeDate();
    view.setDate(LocalDateTime
            .now(ZoneOffset.UTC)
            .minusSeconds(5));

    assertNotNull(view);
    assertEquals("Just now", view
            .getDate());
}
```

```
@Test
void testFutureDate()  {
    NoticeDate view = new NoticeDate();
    view.setDate(LocalDateTime
            .now(ZoneOffset.UTC)
            .plusMonths(5));

    assertNotNull(view);
    assertEquals("Just now", view
            .getDate());
}
```

Tabella 5.2: Casi di test

Caso	`date`	Classi	Esito atteso
TC1	`null`	C_1	`NullPointerException`
TC2	`now() - 3 anni`	C_9	"3 years ago"
TC3	`now() - 11 mesi`	C_8	"11 months ago"
TC4	`now() - 2 settimane`	C_7	"2 weeks ago"
TC5	`now() - 6 giorni`	C_6	"6 days ago"
TC6	`now() - 23 ore`	C_5	"23 hours ago"
TC7	`now() - 59 minuti`	C_4	"59 minutes ago"
TC8	`now() - 5 secondi`	C_3	"Just now"
TC9	`now() + 5 secondi`	C_2	"Just now"

5.3 Testing lato back-end

5.3.1 Test plan: creazione di una issue

5.3.2 Metodo `creaIssue()`

La firma del metodo è la seguente:

```
creaIssue(idCreatore, titolo, descrizione, tipo, priorita, immagine)
```

Da questo metodo abbiamo identificato le seguenti classi di equivalenza:

idCreatore: CE1 = $\{x$ è l'id di un utente interno esistente$\}$ (Valida)
idCreatore: CE2 = $\{x$ non corrisponde a nessun utente$\}$ (Non valida)
tipo: CE3 = $\{x =$ BUG$\}$ (Valida)
tipo: CE4 = $\{x \in \{QUESTION, DOCUMENTATION, FEATURE\}\}$ (Valida)
priorita: CE5 = $\{x \in \{BASSA, MEDIA, ALTA\}\}$ (Valida)
priorita: CE6 = $\{x$ è NULL$\}$ (Valida, la priorità diventa NONE)
immagine: CE7 = $\{x$ è un array di byte$\}$ (Valida)
immagine: CE8 = $\{x$ è NULL$\}$ (Valida, issue senza immagine)
titolo: CE9 = $\{x$ è una stringa non vuota di al massimo 100 caratteri$\}$ (Valida)
descrizione: CE10 = $\{x$ è una stringa non vuota di al massimo 500 caratteri$\}$ (Valida)

Le classi non valide di titolo e descrizione ({x è NULL oppure ""} e {x supera la lunghezza massima}) non sono state considerate: la validazione della richiesta le rifiuta con errore 400 prima che creaIssue venga chiamato.

```
class UtenteInternoServiceTest {

    private final Integer idUtenteCreatore = 7;

    private IssueRepository issueRepository;
    private UtenteInternoRepository utenteInternoRepository;
    private UtenteInternoService utenteInternoService;
    private UtenteNormale utenteCreatore;

    // Setup: crea i mock per le repository e la classe contenente
    // creaIssue, infine crea un utente con id=7
    @BeforeEach
    void setUp() {
        issueRepository = mock(IssueRepository.class);
        utenteInternoRepository = mock(UtenteInternoRepository.class);
        utenteInternoService = new UtenteInternoService(issueRepository,
        utenteInternoRepository);

        utenteCreatore = new UtenteNormale();
        utenteCreatore.setId(idUtenteCreatore);
        when(utenteInternoRepository.findById(idUtenteCreatore))
        .thenReturn(Optional.of(utenteCreatore));
    }

    // CI1 - Utente creatore inesistente: la repository non
    // lo trova e lancia l'eccezione
    @Test
    void creaIssue_creatoreNonTrovato_lanciaEccezione() {
        // precondizione: l'id 99 non corrisponde a nessun utente
        when(utenteInternoRepository.findById(99))
        .thenReturn(Optional.empty());

        // si lancia l'eccezione con questo id
        assertThrows(EntityNotFoundException.class, () ->
                        utenteInternoService
                        .creaIssue(99, "Titolo", "Descrizione",
                                TipoIssue.BUG, Priorita.ALTA, null));

        // controlla che nessuna issue è stata salvata
        verify(issueRepository, times(0)).save(any());
    }

     // CI2 - Issue tipo BUG: viene creato un Bug, non
     // assegnato e non duplicato.
```

```
@Test
void creaIssue_tipoBug_creaBug() {

    utenteInternoService.creaIssue(idUtenteCreatore, "Login non funziona",
    "Errore 500 al login",
            TipoIssue.BUG, Priorita.ALTA, null);

    // assert
    verify(issueRepository).save(any(Bug.class));
    verify(issueRepository).save(argThat(allOf(
            hasProperty("tipo", equalTo(TipoIssue.BUG)),
            hasProperty("assegnatario", is(nullValue())),
            hasProperty("duplicato", is(false)))));
}

 // CI3 - Issue tipo QUESTION
@Test
void creaIssue_tipoQuestion() {

    utenteInternoService.creaIssue(idUtenteCreatore, "Avete provato
    la nuova funzione dell'IDE?",
            "Vorrei qualche delucidazione in merito",
            TipoIssue.QUESTION, Priorita.BASSA, null);

    // assert: salvata un'issue di tipo QUESTION
    verify(issueRepository).save(argThat(hasProperty("tipo",
    equalTo(TipoIssue.QUESTION))));
}

// CI4 - Issue tipo DOCUMENTATION
@Test
void creaIssue_tipoDocumentation() {

    utenteInternoService.creaIssue(idUtenteCreatore, "Cosa restituisce
            questo endpoint?",
            "Manca una documentazione adeguata",
            TipoIssue.DOCUMENTATION, Priorita.MEDIA, null);

    // assert
    verify(issueRepository).save(argThat(hasProperty("tipo",
    sequalTo(TipoIssue.DOCUMENTATION))));
}
```

```
// CI5 - Issue tipo FEATURE
    @Test
    void creaIssue_tipoFeature() {

        utenteInternoService.creaIssue(idUtenteCreatore,
                "Dimenticato di implementare la crittografia dei dati",
                "Feature importante richiesta dal cliente",
                TipoIssue.FEATURE, Priorita.ALTA, null);

        // assert
        verify(issueRepository).save(argThat(hasProperty("tipo",
        equalTo(TipoIssue.FEATURE))));
    }

    // CI6 - Issue con priorità non indicata: vale NONE.
    @Test
    void creaIssue_prioritaNull() {

        utenteInternoService.creaIssue(idUtenteCreatore,
        "Login non funziona", "Errore 500 al login",
                TipoIssue.FEATURE, null, null);

        // assert
        verify(issueRepository)
        .save(argThat(hasProperty("priorita",
        equalTo(Priorita.NONE))));
    }

    // CI7 - Issue con priorità indicata
    @Test
    void creaIssue_prioritaIndicata() {
        utenteInternoService.creaIssue(idUtenteCreatore,
        "Login non funziona",
        "Errore 500 al login",
                TipoIssue.FEATURE, Priorita.MEDIA, null);

        // assert
        verify(issueRepository)
        .save(argThat(hasProperty("priorita",
        equalTo(Priorita.MEDIA))));
    }
```

```
    // CI8 - Regole di servizio valide per ogni issue: imposta data
    // di creazione e stato=TODO
    @Test
    void creaIssue_impostaStatoTodoEData() {

        utenteInternoService.creaIssue(idUtenteCreatore,
        "Login non funziona", "Errore 500 al login",
                TipoIssue.QUESTION, Priorita.ALTA, null);

        // assert
        verify(issueRepository).save(argThat(allOf(
                hasProperty("stato", equalTo(StatoIssue.TODO)),
                hasProperty("data", is(not(nullValue()))),
                hasProperty("utenteCreatore", equalTo(utenteCreatore)),
                hasProperty("titolo", equalTo("Login non funziona")),
                hasProperty("descrizione", equalTo("Errore 500 al login")),
                hasProperty("immagine", is(nullValue())))));
    }

  // CI9 - Issue con immagine presente
    @Test
    void creaIssue_conImmagine() {
        byte[] immagine = {1, 2, 3, 4};

        utenteInternoService.creaIssue(idUtenteCreatore,
        "Login non funziona", "Errore 500 al login",
                TipoIssue.BUG, Priorita.ALTA, immagine);

        // assert
        verify(issueRepository).save(argThat(hasProperty("immagine",
         equalTo(immagine))));
    }
}
```

CI1 (riga 57): creaIssue(99, "Titolo", "Descrizione", BUG, ALTA, NIL), copre CE2, CE3, CE5, CE8, CE9, CE10
CI2 (riga 72): creaIssue(7, "Login non funziona", "Errore 500 al login", BUG, ALTA, NIL), copre CE1, CE3, CE5, CE8, CE9, CE10
CI3 (riga 87): creaIssue(7, "Avete provato la nuova funzione dell'IDE?", "Vorrei qualche delucidazione in merito", QUESTION, BASSA, NIL), copre CE1, CE4, CE5, CE8, CE9, CE10
CI4 (riga 99): creaIssue(7, "Cosa restituisce questo endpoint?", "Manca una documentazione adeguata", DOCUMENTATION, MEDIA, NIL), copre CE1, CE4, CE5, CE8, CE9, CE10
CI5 (riga 111): creaIssue(7, "Dimenticato di implementare la crittografia dei dati", "Feature importante richiesta dal cliente", FEATURE, ALTA, NIL), copre CE1, CE4, CE5, CE8, CE9, CE10
CI6 (riga 123): creaIssue(7, "Login non funziona", "Errore 500 al login", FEATURE, NIL, NIL), copre CE1, CE4, CE6, CE8, CE9, CE10

CI7 (riga 134): creaIssue(7, "Login non funziona", "Errore 500 al login", FEATURE, MEDIA, NIL), copre CE1, CE4, CE5, CE8, CE9, CE10
CI8 (riga 144): creaIssue(7, "Login non funziona", "Errore 500 al login", QUESTION, ALTA, NIL), copre CE1, CE4, CE5, CE8, CE9, CE10
CI9 (riga 161): creaIssue(7, "Login non funziona", "Errore 500 al login", BUG, ALTA, 1, 2, 3, 4), copre CE1, CE3, CE5, CE7, CE9, CE10

5.3.2.1 Criterio di copertura delle classi di equivalenza

Come criterio di copertura si è adottato l'Each Choice Coverage (ECC): ogni classe di equivalenza di ogni parametro è rappresentata da almeno un caso di test. I test da CI1 a CI9 coprono così tutte le classi individuate.

5.4 Test plan: creazione di una issue

5.4.1 Obiettivo

Verificare che un utente interno (amministratore o utente normale) possa segnalare un'issue e che il sistema applichi le seguenti regole di servizio: titolo e descrizione obbligatori, stato iniziale TODO, priorità NONE se non indicata e creazione di un oggetto bug solo per le issue di tipo BUG. Verificare inoltre che gli utenti non autorizzati vengano respinti.

5.4.2 Oggetti del test

- L'endpoint POST /api/issues (che corrisponde a IssueController.creaIssue), che controlla il ruolo dell'utente e la validità dei dati ricevuti.
- La classe CreaIssueRequest, che contiene i vincoli sui dati in ingresso (campi obbligatori e lunghezze).
- Il metodo UtenteInternoService.creaIssue, che applica le regole di creazione.

5.4.3 Ambiente di testing

Back-end e database MySQL avviati ed operativi; nel database sono presenti l'amministratore di default, un utente normale e uno stakeholder. Prima di ogni caso di test l'utente indicato ha effettuato il login e la richiesta contiene il suo cookie di sessione.

Tranne dove indicato, il corpo della richiesta è sempre:

```
{ "titolo": "Login non funziona", "descrizione": "Errore 500 al login", "tipo": "BUG",
"priorita": "ALTA", "immagine": null }
```

5.4.4 Criteri di superamento

Il test plan è superato se tutti i casi di test producono l'output e le postcondizioni attese.

Id	*Precondizioni*	*Input*	*Output effettivo*	*Postcondizioni*
TP1	Nessun login	Corpo di riferimento	401 Unauthorized, “Utente non autenticato”	Nessuna issue creata
TP2	Login come stakeholder	Corpo di riferimento	403 Forbidden, ruolo non ammesso, “Operazione non permessa”	Nessuna issue creata
TP3	Login come utente normale	Corpo di riferimento	201 Created	Bug creato (nel db righe in Issues e in Bugs). UtenteCreatore = l’utente loggato
TP4	Login come amministratore	Corpo di riferimento	201	Bug creato (nel db righe in Issues e in Bugs). UtenteCreatore = l’utente loggato
TP5	Login come utente normale	tipo = FEATURE	201	Riga creata solo in Issues
TP6	Login come utente normale	priorità assente	201, priorità NONE	Issue creata
TP7	Login come utente normale	titolo assente	400 Bad request, richiesta non valida, “Il titolo è obbligatorio”	Nessuna issue creata
TP8	Login come utente normale	titolo di 1 carattere	201	Issue creata
TP9	Login come utente normale	titolo di 100 caratteri	201	Issue creata
TP10	Login come utente normale	titolo di 101 caratteri	400 , “Il titolo può avere al massimo 100 caratteri”	Nessuna issue creata
TP11	Login come utente normale	descrizione di 500 caratteri	201	Issue creata
TP12	Login come utente normale	descrizione di 501 caratteri	400, “La descrizione può avere al massimo 500 caratteri”	Nessuna issue creata
TP13	Login come utente normale	tipo assente	400, “Il tipo è obbligatorio”	Nessuna issue creata
TP14	Login come utente normale	tipo = "PIPPO" oppure priorità = "URGENTE"	400	Nessuna issue creata
TP15	Login come utente normale	immagine di 2 MB	201, con immagine	Issue creata
TP16	Login come utente normale	JSON non valido	400, “Richiesta non valida”	Nessuna issue creata